

Engenharia Reversa na Engenharia de Software

O que é?

“É o processo de análise de um determinado sistema com dois objetivos em mente:

- 1) identificar os componentes do sistema e os seus inter-relacionamentos; e,*
- 2) criar representações do sistema em outra forma ou em níveis mais altos de abstração”.*

Engenharia Reversa normalmente é empreendida com o objetivo de reprojetar um sistema para melhorar a sua manutenção ou produzir uma cópia de um sistema sem acesso às informações de seu projeto original.

É uma área de interesse emergente principalmente pelo volume de sistemas legados que precisam evoluir. Diz-se Engenharia Reversa porque, ao contrário da engenharia tradicional, partimos do produto para a sua definição. Portanto, antes de evoluir um determinado sistema de software, faz-se necessário reverter-lo à descrições mais abstratas com o objetivo de facilitar nossa compreensão do que o sistema é, como funciona e como não funciona, para então modificá-lo. Como se trata de um processo, a Engenharia Reversa pode ser aplicada a cada um dos três aspectos principais de um sistema: dados, processo e controle.

Suas Origens

É notável observar a máxima que diz que a coisa mais complexa fabricada pelo ser humano não é o mais moderno dos aviões ou dos computadores, mas sim o software. A Engenharia Reversa em software tem seus primórdios na Engenharia Reversa em hardware, que tem como objetivo a obtenção dos segredos de projeto e de produção de um competidor. Em software é bem similar mas na maioria dos casos o programa a sofrer a Engenharia Reversa não pertence a um competidor, mas sim a empresa que está realizando-a.

Reengenharia de Software

A Engenharia Reversa não é uma parte isolada na Engenharia de Software, está contida num contexto mais amplo : a Reengenharia de Software.

Antes de mais nada, por que fazer a Engenharia Reversa ? Do que adianta obter a descrição abstrata e documental de um programa ? Se um software está desatualizado, por que perder tempo tentando entender um código obscuro ? Não seria melhor construir um sistema novo em folha usando novas tecnologias e paradigmas atuais de desenvolvimento ?

A Reengenharia em si não se restringe somente ao software. Ela pode ser aplicada numa empresa com o intuito de enxergá-la como uma grande rede de processos interconectados onde cada processo seria entendido e reestruturado de forma a ser executado da melhor

forma possível. A forma como o processo é executado atualmente é abandonada e a forma considerada ideal é adotada.

Cada processo teria como foco um conjunto de clientes, sejam processos internos ou clientes externos à empresa. Em sua última instância, seria levar o conceito de linha de montagem às atividades administrativas de uma organização e aplicar os conceitos já conhecidos no chão das fábricas para os escritórios e além. Ao contrário de uma única linha de montagem, várias linhas existiriam. O problema reside em garantir o fluxo de cada linha com o objetivo de maximizar a satisfação de cada cliente. Uma sinergia que garantiria que o todo obtivesse ganhos maiores que a soma de suas partes.

Por se tratar de uma série de processos, o ajuste fino e contínuo de cada é vital para o funcionamento da organização como um todo. Através de medidas, cada processo poderia ser regulado o mais prontamente possível. A disponibilidade de tais métricas também é outro fator chave. O próprio processo deve ser capaz de fornecer suas métricas, ser avaliado e ter o seu próprio *feedback*, ou seja, a sua própria regulação. A rapidez com que é regulado e com que as métricas são obtidas são outros fatores importantes.

De fato trata-se de uma visão bem diferente da estrutura funcional rígida existente em algumas empresas. O uso da Tecnologia de Informação (TI) surge como um viabilizador desse tipo de visão processual interativa, se tornando por isso no alavancador da reengenharia. O fato de se usar a TI por si só não indica que haja essa visão de processo. Muitas empresas já estão informatizadas apesar de possuírem uma estrutura ainda baseada em modelos funcionais arcaicos.

Alguns benefícios da reengenharia :

- redução de custos;
- melhoramento da performance dos processos;
- aproveitamento dos chamados *breakthroughs**

** entendidos como sendo algo que permite alcançar uma vantagem competitiva significativa e percebida pelo mercado como um elemento diferenciador, o que normalmente resulta num acréscimo sustentado por uma fatia desse mercado.*

Por que devemos *reengenheirar* um software?

A Reengenharia de Software poderia ser conceituada da seguinte forma:

"É o exame de um sistema para reconstituí-lo em uma forma nova e a implementação subsequente dessa nova forma. "

1) Manutenção do Software

Um cenário muito comum é uma aplicação que tem servido uma atividade de negócios durante anos (10 à 15 anos). Durante esse tempo ele tem sido corrigido, adaptado e

aprimorado muitas vezes. Porém as boas práticas da Engenharia de Software foram postas de lado. Agora a aplicação está instável, porém em uso. Durante cada processo de mudança uma gama infindável de efeitos colaterais surgia. Ainda assim a aplicação tem que continuar evoluindo.

Softwares de difícil manutenção não é um problema novo. A ênfase da Reengenharia de Software tem sido nesses softwares, cuja facilidade manutenção tem sido posta em segundo plano por décadas.

A necessidade de mudanças é uma constante em todo trabalho de produção do software. São inevitáveis quando sistemas baseados em computadores são construídos. Mecanismos têm que ser desenvolvidos para avaliar, controlar e executar modificações de uma forma que o impacto no sistema seja o menor possível.

2) Controle das Informações

O ambiente internacional competitivo a partir de meados dos anos 90 demanda que uma organização faça o melhor de seus recursos. Recursos que podem contribuir diretamente com a missão da organização são de valor significativo. Um recurso que pode ser utilizado para vantagens estratégicas e táticas é inestimável.

Na era da informação que há pouco entramos, dados são recursos inestimáveis. Fontes de dados existem em muitas formas em uma organização. Algumas estão organizadas na forma de bancos de dados ou arquivos. Outras se encontram em alguns tipos de estruturas de dados enigmáticas em sistemas projetados há muito tempo cujos projetistas já estão aposentados. Outros tipos de dados encontram-se em pequenas aplicações que alguém lançou mão há poucos anos como um remendo temporário e tem sido usadas desde então. Há ainda fontes de dados furtivas que nem mesmo as organizações sabem que possuem.

Da mesma forma que a Reengenharia de Processos tenta modificar os processos de uma organização, a Reengenharia de Software procura modificar o software como o objetivo de que ele se enquadre num novo ambiente processual, tendo o seu valor expandido ou mesmo extrapolado. O software passa de mero coadjuvante nas atividades cotidianas para ser um dos atores do processo. Ele passa a fornecer métricas de como o processo se desenvolve e participar no seu controle. Além disso, torna-se dinâmico pois foi remodelado para suportar mudanças, enquadrando-se nos ambientes competitivos das organizações empresariais.

Modelo de Reengenharia de Processos

Reengenheirar consome tempo e dinheiro absorvendo recursos que poderiam ser empregados em atividades com necessidades mais imediatas. Pode-se tornar uma atividade que dura meses ou alguns anos. Uma estratégia que coordene esse atividade se faz necessária. Tal estratégia exige que a reengenharia seja considerada como uma atividade de reconstrução.

O modelo do processo de Reengenharia de Software define seis atividades:

- Análise de inventário : é o levantamento do inventário de todas as informações sobre as aplicações, não importando o meio onde estejam. Ele deve fornecer uma descrição detalhada sobre cada aplicação ativa, classificando as informações colhidas de acordo com a sua importância para o negócio, longevidade, estado de manutenção e algum outro critério local importante que possa servir como base para avaliar a necessidade uma reengenharia.
- Reestruturação de Documentação : documentação escassa é uma característica que muitos sistemas em funcionamento possuem há muito tempo. Logo, a reconstrução completa da documentação é de vital importância para o entendimento do sistema.
- Engenharia Reversa : importante atividade já discutida acima que reverte um produto em suas definições mais abstratas de desenvolvimento com o objetivo de facilitar nossa compreensão do que é o sistema, como funciona e como não funciona para então poder ser modificado de acordo com as necessidades apontadas pela Reengenharia.
- Reestruturação de código : alguns sistemas legados possuem uma arquitetura relativamente sólida mas módulos individuais foram codificados em uma forma que os torna difíceis de serem entendidos, testados e mantidos. Violações da programação estruturada devem ser anotadas e então reestruturadas. O código resultante deve ser revisado e testado para assegurar que nenhuma anomalia foi introduzida. A documentação interna deve ser atualizada.
- Reestruturação de dados : em sistemas *arquiteturalmente* fracos, será difícil adaptar os dados e aprimorá-los. De fato, para muitos aplicações, a arquitetura do dados tem mais a ver com a disponibilidade a longo prazo do que o código fonte em si. Ela ocorre em um nível de abstração mais baixo do que a reestruturação de código. Será dissecada e outros modelos de dados serão definidos se necessário.
- Forward Engineering : é também chamada de renovação. Não apenas recupera as informações do software existente mas o altera e o reconstitui num esforço para aprimorar sua qualidade. O software *reengenheirado* implementa funções do sistema existente e também adiciona e/ou melhora a sua performance nas funções que já existiam e que foram mantidas.

Em alguns casos, essas atividades ocorrem numa sequência linear, mas nem sempre é o caso. De uma forma mais completa, a atividade de reengenharia pode ser vista como um ciclo, o que não é de espantar já que é um modelo de desenvolvimento de software. Porém, um diferencial é que não há especificamente uma atividade inicial.

Após a fase da Engenharia Reversa, as alterações do código, podem ocorrer em três níveis distintos de abstração:

- ao nível físico : alteração da base tecnológica de suporte;
- ao nível lógico : alteração do software;

- ao nível conceptual : alteração da forma de funcionamento dos processos inerentes à aplicação em causa.

A reengenharia resume-se aos seguintes passos :

- documentar os softwares atuais;
- melhorar a leitura do código ;
- redesenhar as bases de dados ;
- alterar a plataforma de hardware ;
- converter linguagens ;
- adicionar novas funcionalidades e/ou capacidades;
- facilitar os processos de manutenção e fazer evoluir os softwares num ambiente CASE (tecnologia que inclui ferramentas de análise, projeto, desenvolvimento e análise de performance).

Engenharia Reversa

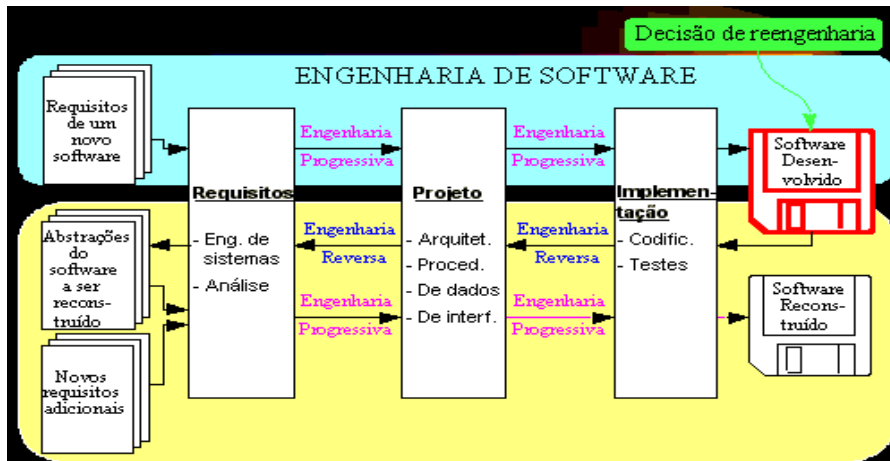
A atividade de manutenção de software é reconhecidamente uma fase problemática. Tais problemas são causadores de custos substanciais quando comparados com os custos das outras fases do ciclo de vida do software. A facilidade de manutenção (manutenibilidade), caracterizada principalmente pelo entendimento do software, está fortemente relacionada à disponibilidade de documentação sobre o software.

No entanto, na maioria das vezes, a documentação é inexistente, incompleta e/ou desatualizada devido à fatores como o software ser muito antigo ou à falta de documentação de atualizações nas modificações do software.

Diversos fatores contribuíram para motivar o aparecimento da Engenharia Reversa dentro de um contexto da Reengenharia de Software. Entre eles estão os seguintes fatores :

- software tem vida limitada e alterações na sua estrutura podem degenerá-la;
- software deve evoluir continuamente ou tornar-se-á menos útil no mundo real;
- software legado que executa tarefas úteis mas que foi desenvolvido com técnicas obsoletas que dificultam a sua manutenção e evolução;
- problemas nos softwares legados : desestruturação e dificuldade de entendimento do código, documentação desatualizada, programadores que não desenvolveram o produto e efeitos colaterais causados por alterações;

Todos esse fatores causam o aparecimento de um dilema na decisão sobre o futuro de um software : manter, descartar, melhorar ou reconstruir ? Caso a opção escolhida seja a última (reconstrução), a tarefa será executada através da Engenharia Reversa no contexto de uma Reengenharia do software em questão.



Muitas vezes a opção de descartar o software e desenvolver um novo não é aceita por se reconhecer o valor do acúmulo de experiências e informações obtidas durante anos e que estão embutidas no software. Além disso, muitas vezes é economicamente inviável descartar o alto investimento financeiro atribuído ao software sendo assim a reconstrução a atividade escolhida.

Nesse sentido, a Engenharia Reversa tem por objetivo principal contribuir, primeiramente, no entendimento e, posteriormente, na modificação e revalidação do software, aumentando assim a manutenibilidade do mesmo. Isto é feito através de um processo de análise que procura criar representações dos programas em uma forma mais clara ou em um nível mais alto de abstração que o código fonte, identificando os componentes desse sistema e os seus inter-relacionamentos.

Um software é desenvolvido baseado em três níveis de abstração distintos (ciclo de vida do software), caracterizando a sequência de etapas da Engenharia Progressiva :

- Engenharia de Sistemas (*porque ?*)
- Análise de Requisitos (*o que ?*)
- Desenvolvimento, que abrange as fase de Projeto, Codificação e Testes (*como ?*)

A Engenharia Reversa é um processo que tem um fluxo na direção contrária da Engenharia Progressiva visando a produção, preferencialmente de forma automática, de documentos que ajudem a aumentar o conhecimento geral dos sistemas de software.

A Forward Engineering é uma atividade que, apoiada na documentação detalhada obtida durante o processo de Engenharia Reversa, reconstrói o software de maneira que o software reconstruído tenha maiores facilidades de reutilização, manutenção, teste e controle de qualidade de software.

As duas etapas de Engenharia Reversa e Forward Engineering fazem parte da Reengenharia do Software (vide figura acima).

A figura abaixo, relaciona os três processos de engenharia : Progressiva, Reversa e a fase de reconstrução (Forward Engineering). Deve-se notar que, após a obtenção das abstrações e documentações pela Engenharia Reversa, a reconstrução pode ser efetuada adicionando-se novos requisitos ou alterando os que já existiam.

São definidas duas categorias de Engenharia Reversa : a Visualização de Código e o Entendimento do Programa.

- **Visualização de Código ou Redocumentação** : é a forma mais simples e mais antiga da Engenharia Reversa. A intenção é recuperar a documentação que já existiu, ou que deveria ter existido, sobre o sistema. A ênfase é a criação de visões adicionais, especialmente visões gráficas, que não foram criadas durante o processo original de Engenharia Progressiva.
- **Entendimento do Programa ou Recuperação de Projeto** : é o conhecimento do domínio das informações externas. As deduções são adicionadas às observações feitas sobre o sistema através do exame do mesmo de modo a obter informações com nível mais alto de abstrações.

Em relação à categoria de Entendimento do Programa, podemos dividir a Engenharia Reversa em três etapas de entendimento : Processo, Dados e Interfaces dos Usuários.

1. Entendimento do Processo

A primeira atividade real da Engenharia Reversa se inicia com uma tentativa de entender e então extrair as abstrações da funcionalidade do processo representadas através de uma combinação do código fonte, documentação existente, experiências pessoais e conhecimentos gerais sobre o problema e o domínio da aplicação

Para extrair tais abstrações, o código é analisado em diversos níveis de abstrações : sistema, programa, módulo, padrão e implementação.

A funcionalidade do sistema como um todo deve ser compreendida antes da obtenção de maiores detalhes, o que ajuda a compreender possíveis interações que o sistema possua com outras aplicações.

Cada programa que constitui o sistema representa uma abstração funcional em um nível diferente de detalhe sendo criado um modelo que mapeia as interações entre eles. Cada programa é composto de um conjunto de módulos que executa uma função específica que representa uma abstração funcional criando-se também modelos que mapeiam suas interações.

Essa fase torna-se mais complexa quando o engenheiro procura dentro dos módulos por seções do código que representem um padrão (*pattern*) de procedimento genérico. Geralmente, todos os módulos possuem pelo menos três seções : uma que

prepara os dados para serem processados, outra que executa o processamento propriamente dito e, por fim, uma que prepara os resultados para serem exportados para fora do módulo. Dentro dessas seções, é possível se encontrar alguns padrões.

2. Entendimento dos Dados

Também ocorre em diferentes níveis de abstração : ao nível do programa (estruturas internas de dados do programa) e ao nível do sistema (estruturas globais de dados).

- Estruturas Internas de Dados do Programa

É focada na definição de classes de objetos (paradigma de Orientação à Objetos) a partir da análise das estruturas de dados internas de um programa.

Bons indicadores de classes são obtidos examinando-se o código do programa com a intenção de agrupar algumas de suas variáveis e verificando-se as estruturas compostas de dados como registros, arquivos, listas, vetores e árvores. Além disso, é necessário avaliar a interação das variáveis internas com as estruturas de dados globais para que sejam definidas também possíveis classes que implementem essa interação.

A Engenharia Reversa também identifica componentes para futura reutilização na construção de um software, outra característica importante e fundamental do conceito de Orientação à Objetos.

- Estrutura de Banco de Dados

Talvez essa seja uma das mais difundidas e usadas técnicas de Engenharia Reversa. Com certeza, a maior necessidade de alteração dos sistemas legados diz respeito a forma como os dados são armazenados. Logo, é necessário extrair dos dados os seus modelos conceituais para que eles possam ser acomodados aos novos paradigmas de gerenciamento de banco de dados.

Para representar um esquema de banco de dados em um outro esquema, é necessário a definição dos objetos de dados e a forma como eles se relacionam, ou seja, obter o modelo conceitual de dados. Com o modelo em mãos é possível alterar o modelo físico de um banco de dados de acordo com as necessidades apontadas pela Reengenharia.

-

1. Entendimento das Interfaces dos Usuários

Essa etapa também é uma das atividades mais comuns no processo de Reengenharia.

Quando as interfaces dos usuários começam a se tornar desatualizadas é necessário um novo desenvolvimento dessas interfaces.

Porém, antes disso, uma atividade de Engenharia Reversa deve ocorrer. Para que possamos entender completamente uma interface do usuário existente, a estrutura e comportamentos dessa interface devem ser especificadas.

Um conjunto de três perguntas básicas que podem ser feitas quando se pretende realizar a Engenharia Reversa de uma interface do usuário são mostradas abaixo :

- Quais as ações básicas que a interface deve executar ? (ex: ao mouse ser clicado num determinado campo ou botão ou quando uma determinada tecla do teclado é pressionada)
- Qual o comportamento esperado da interface em resposta às ações executadas ?
- Qual o conceito de equivalência de interfaces é relevante , ou seja, a sua substituição ?

Um modelo comportamental pode ser criado para representar as respostas da primeira e segunda perguntas acima. Muitas das informações necessárias para a criação desse modelo são obtidas pela observação da manifestação externa das interfaces existentes. Porém, informações extras devem ser extraídas do código do sistema.

-

Métodos e Ferramentas de Apoio à Engenharia Reversa

Nos últimos anos, há um crescente reconhecimento da importância da Engenharia Reversa tanto no campo acadêmico como no ambiente de produção, tendo resultado na apresentação de diversas pesquisas abordando diferentes métodos e técnicas.

Nesse escopo, procura-se identificar ferramentas já existentes ou até mesmo especificar novas ferramentas que possam servir de apoio aos passos mais trabalhosos da abordagem.

A Engenharia Reversa de sistemas representa um problema de grande complexidade computacional no qual é constante a necessidade de interagir com humanos para complementar lacunas do conhecimento embutido em software legados. Por outro lado, é imprescindível o auxílio de ferramentas computacionais para que se tente ao máximo automatizar as diversas etapas do processo.

Dois tipos básicos de ferramentas :

- Ferramentas de Visualização de Código : fornecem visões alternativas para o usuário entender o sistema.
- Ferramentas de Entendimento de Programa : objetivam entender o sistema, sendo a forma mais crítica de Engenharia Reversa porque tenta imitar o raciocínio humano na busca do entendimento.

Alguns exemplos de ferramentas e métodos de Entendimento de Programa :

- Ferramentas para recuperação dos modelos da base de dados : Dbmain, ErWin, PowerDesigner

- Ferramentas para recuperação da estrutura de chamadas : Rigi
- Métodos para engenharia reversa orientada a objetos : Fusion/RE.
- Ferramentas para entendimento das Interfaces dos Usuários : *GeneXus Corporate*, **ferramenta** I-CASE baseada no conhecimento que captura e descreve a visão dos eventos de cada usuário consolidando-as de forma automática de modo a produzir uma base de conhecimentos.

Alguns métodos para recuperação dos modelos da base de dados :

CaSa84, Davi90, MaMa90, Ji90, BCN92, ShSh93, Ande94, CBS94, Joha94, PrB194, Peti94, Sign94, Peti95, Peti96, Mfou97. Esses métodos podem ser agrupados nas seguintes categorias:

- Categoria 1: Essa categoria contém os métodos de engenharia reversa [BCN92, ShSh93, CBS94] que classificam as relações. As relações são classificadas para depois serem transformadas em um esquema conceitual. O método de Chiang [CBS94] além de classificar as relações, classifica também os atributos. Os métodos de Chiang [CBS94] e de Batini e outros [BCN] possuem características semelhantes pois, utilizam nomes consistentes para os atributos. O método de Chiang [CBS94] utiliza além do esquema de dados, também as instâncias de dados.
- Categoria 2: Essa categoria contém os métodos de engenharia reversa [CaSa84, MaMa90, Joha94, PrB194, Peti96] que tem como característica básica a utilização das dependências de inclusão para descobrir os tipos relacionamentos e também os tipos entidades. Os métodos dessa categoria são semelhantes apesar de terem algumas particularidades.
- Categoria 3: Essa categoria contém os métodos de engenharia reversa [Peti94, Peti95] que utilizam as consultas SQL para achar os tipos entidades e os tipos relacionamentos. As instâncias de dados extraídas das consultas SQL são analisadas.
- Categoria 4: Essa categoria contém os métodos de engenharia reversa [Ande94, Ji91] que utilizam um grafo para descobrir os tipos entidades e os tipos relacionamentos. Os métodos de Andersson [Ande94] e de Ji [Ji91] utilizam as dependências de inclusão para construir o grafo. O método de Andersson [Ande94] utiliza ainda as consultas SQL para ajudar a construir o grafo.
- Categoria 5: Essa categoria contém o método de engenharia reversa [Mfou97] que utiliza um conjunto de formulários para descobrir os tipos entidades e os tipos relacionamentos. O conjunto de formulários é gerado através de consultas SQL. A descoberta dos tipos relacionamentos entre os tipos entidades é feita através da observação das instâncias dos dados contidas no conjunto de formulários.
- Categoria 6: Essa categoria contém o método de engenharia reversa [Davi90] que utiliza apenas de nomes consistentes para os atributos para achar os tipos

relacionamentos. Não são considerados nesse método de Davis [Davi90] as dependências de inclusão.

- Categoria 7: Essa categoria contém o método de engenharia reversa [Sign94] que não apresenta algoritmos para transformar o esquema relacional em um esquema conceitual. Esse método de Signore [Sign94] apresenta apenas informações (i.e. texto DDL, esquema relacional, dependências de inclusão, consultas SQL) que podem dizer qual tipo de objeto (i.e. tipo relacionamento ou tipo entidade) será extraído dessa informação.

Conclusões

Em relação a tudo que foi explicitado acima sobre a Engenharia Reversa, é deixado de forma clara a noção de que ela não ocorre isoladamente e sim dentro de um processo de Reengenharia.

Tendo a Tecnologia de Informação como seu principal elemento alavancador, a Reengenharia de Software torna-se essencial dentro desse contexto pois é extremamente necessário entender de forma completa todos os sistemas da empresa.

Logo, dentro do contexto de Reengenharia de Software, a Engenharia Reversa passa a ter papel fundamental pois gera documentação suficiente para o entendimento dos sistemas e para uma possível reconstrução dos mesmos baseados em novos paradigmas e conceitos mais atualizados.

Características interessantes da Engenharia Reversa

- Tópicos da Engenharia de Software com as quais a Engenharia Reversa se confronta
- Engenharia Reversa X Manutenção : está fortemente relacionada com a manutenção de software já que as atividades de manutenção fornecem a motivação para muitas ferramentas de Engenharia de Software.
- Engenharia Reversa X Reuso de Componentes : prestam apoio à essa etapa da Orientação à Objetos pois ajudam na definição de componentes possíveis de serem reutilizados.
-
- Engenharia Reversa X Garantia de Qualidade de Software : a Engenharia Reversa pode ser considerada como um conjunto de atividades complementares a cada fase do ciclo de vida podendo ser aplicada desde o seu início como uma base para fornecer garantia de qualidade de software.
- Questões Econômicas : como a fase de manutenção é a motivadora do processo de Engenharia Reversa e é a fase mais custosa do ciclo de vida de um software, um melhor entendimento do software facilita a atividade de manutenção e consequentemente causa o aumento da produtividade.

- Questão Legal : a aplicação da Engenharia Reversa infringe a lei da propriedade intelectual ? Essa questão é difícil de ser respondida e pode gerar árduas discussões. Mas, em tempos de estimulação de código aberto (*open sources*), o processo passa a ser mais bem aceito dentro dessa visão.
- Uma outra característica importante a ser salientada é que a Engenharia Reversa não é limitada à área de desenvolvimento de softwares. Ela é extensível à todas as áreas sendo bastante utilizada em ambiente industrial, não só na forma da possibilidade de reconstruir um produto de melhor qualidade e menor manutenção mas também no que diz respeito a competitividade no mercado, onde produtos dos competidores seriam analisados para que se descobrisse como foram projetados e produzidos.

Bibliografia

IEEE Computer Society

TCSE Committee on Reverse Engineering and Reengineering

<http://www.tcse.org/revengr/>

Obs: TCSE - Technical Council on Software Engineering

Ferramentas Case

<http://www.qucis.queensu.ca/Software-Engineering/vendor.html>

Empresa que realiza engenharia reversa em hardware

<http://www.chipworks.com/home.html>

Reverse Engineering and System Renovation

<http://adam.wins.uva.nl/~x/reeng/REanno.html>

Georgia Tech's Reverse Engineering Group

<http://www.cc.gatech.edu/reverse/>

Data Reverse Engineering Reengineering Bibliography

Livro de cabeceira:

Data Reverse Engineering:

Slaying the Legacy Dragon

<http://www.isy.vcu.edu/~paiken/publications/dre/drebib.htm>